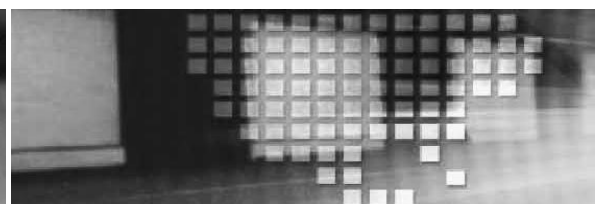
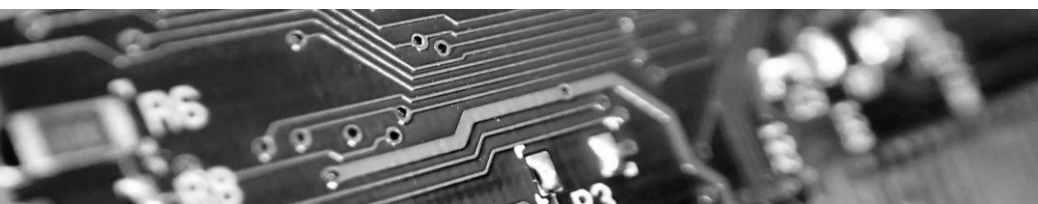




Smart Meter Home Application



Version:

I

Authors:

Piter Garcia
Eric Stravinskas
Will Brennan

FINAL REPORT & INTRUDUCTION MANUAL

TABLE OF CONTENTS

ABSTRACT	1
INTRUDUCTION.....	2
3.0 PROJECT PRIORITIES	3 , 4, 5
4.0 GENERAL INFORMATION	6
4.1 Overview	6
4.2 Components.....	6
4.3 Background Information.....	7
5.0 SYSTEM SUMMARY.....	8
5.1 System Overview.....	8
5.2 System Configuration	9
5.3 Bill of Materials.....	10
6.0 GETTING STARTED	11
6.1 Enabling XBee.....	11, 12, 13, 14
6.2 Setting Up a Network.....	15
6.3 API Mode.....	16, 17, 18
6.4 Digi XBee Smart Plug.....	19
7.0 METHODS & PROCEDURES.....	20
7.1 Hardware (Construction) Phase	20
7.2 Using SKXBee as a Coordinator	21,22
7.3 Modification	23
7.4 Input Procedures and Expected Output.....	24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34
8.0 SCHEDUAL OF PROGRESS.....	35
8.1 First Quarter	35,36
8.2 Second Quarter	36, 37
8.3 Third Quarter	37, 38, 39

10.0 RESULTS	40
11.0 ETHICAL CONCERNS.....	41, 42
12.0 FUTURE CONSIDERATION	43
CONCLUSION	44
PROJECT REFERENCES.....	45

ABSTRACT

ABSTRACT

Home Automation is one of the most famous application profiles that is being researched and implemented by many companies around the world. ZigBee is the global and rapid grow standard of Wireless Sensor Network. This allows the smart meter to communicate with several electrical appliances within the same network. The ZigBee is built on IEEE 802.15.4 radio and MAC and it is the only standard (until now) that support wireless mesh network. Within ZigBee standard, numerous “application profiles” define the standard ZigBee devices that could be used to communicate with each other.

The goal of our project is to design a Smart Meter Application using ZigBee. The Home Automation Smart Meter Application will be able to monitor the amount of energy consumed by users. This application places ZigBee module into our daily used devices such as lamp, radiator, ventilation, and others. The Smart Meter Application monitors every device by judging from the read out from the sensor. Simple devices like those mentioned above could easily be combined together to form a large and complex network within a home. This network will not only make life more comfortable but also save energy dramatically. Likewise, users will be able to see how much energy is being used during peak hours when the cost of energy is high. Therefore, the user will be allowed to manage their consumption conveniently and in detail. The amount of energy used by specific appliances will be monitored and when necessary the appliance can be shut off. The Smart Meter Application will also be able to automatically shut off an appliance when too much energy is being used. In brief, the smart meter application makes it possible for users to conserve electricity and save money.

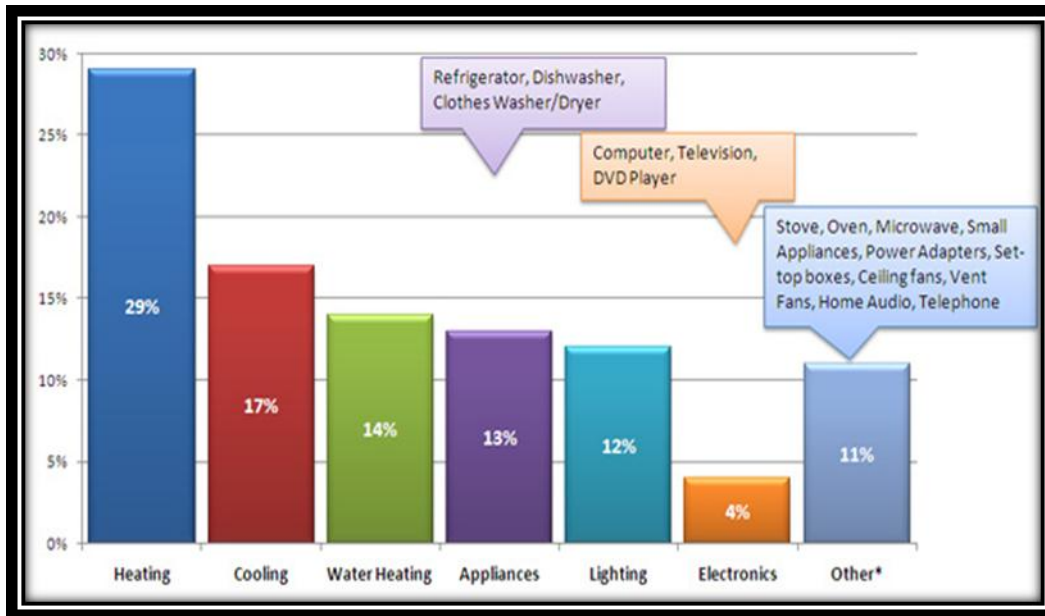
INTRODUCTION

Introduction

The main goal of the Smart Meter Application is to create a network in a home or industry that will link electronic devices to save energy and money while protecting all electronic devices within the network. The application will be most beneficial when the cost of consuming energy is high. First, the home automation application will be able to monitor the amount of energy that users consume by placing a ZigBee module into our daily used devices such as lamp, radiator, ventilation, and others. Second, a smart meter will monitor every electrical device and combine them to form a large and complex network within a home or industry. In this way, the smart meter will turn off any device when needed as well as when too much energy is used. Third, the user will be able to manage their consumption conveniently and in detail. Finally, our Smart Meter Application will not only make life more comfortable but also save energy and save money while protecting our own environment.

3.0 – Project Properties

- PROBLEMS:



Home Energy Waste Chart

The average home in the US spends about \$2,200 a year on electricity. The above diagram shows how much money is spent in specific areas. About half the money is spent while heating and cooling homes when no one is home. If the household is using our smart meter application, this could be avoided in a variety of ways. The homeowner could set a specific temperature that they want the house to be and when that is reached it would turn off. Another way is setting the system to turn on and off at certain times of day. Of course the system can also turn off other things, such as lighting or televisions, when no one is in the room using sensors that are built into the smart plugs.

- SOLUTIONS:

Inspection: Taking a few minutes once a month to examine your appliances to reveal any possible problems. Check for electrical cords cracks and also any other obvious damage.

Batteries: It is helpful to keep a list to let you know when you last replaced your appliances batteries.

Checking Surge Protectors: A wonderful tool to keep your appliances from getting damaged through a power surge; this is very important when you are talking about things like televisions and computers. Making sure that they are working properly can spare you the loss of the appliance from a power surge.

Upgrades: Sometimes we rely upon an appliance that is outdated and consuming a great deal of energy. It can save us money on electrical bills by replacing them with a more efficient model.

Checking Outlets: It is important to periodically make sure that you outlets are working correctly, because if one stops working or is faulty it can affect your appliance.

Turning Off Appliances: Making sure we turn off something, like a computer that is not in use, can save energy and cut down on possible problems from it being on for too long of a period, when not in use.

What if we had one only electronic device that does the above solution methods for us?

The Smart Meter Home Application based on a ZigBee network can do each one of these functions while saving the users energy, money and protecting the electronic appliances. The smart meter network, when placed into our daily used electronic appliances, can easily combine them together to form a simple network within a home and use them efficiently.

4.0 – General Information

4.1 Overview

- System environment
 - This network set up is affordable and friendly to the environment.
- ZigBee Network Technology
 - It is one of the simplest and cheapest technologies with the best innovation in automation.
- Conserve and Control Power
 - It conserves electronic devices consumed energy while protecting them from overloading.
- Home Automation & Power Control
 - Name assigned to the team in charge of this network setup's instruction manual.
- Coordinator, Routers, and End Devices
 - Main Components needed to build a network with ZigBee technology.

4.2 Components

Coordinator:

The coordinator maintains overall network knowledge. It is the most sophisticated of the three types and requires the most memory and computing power.

Routers:

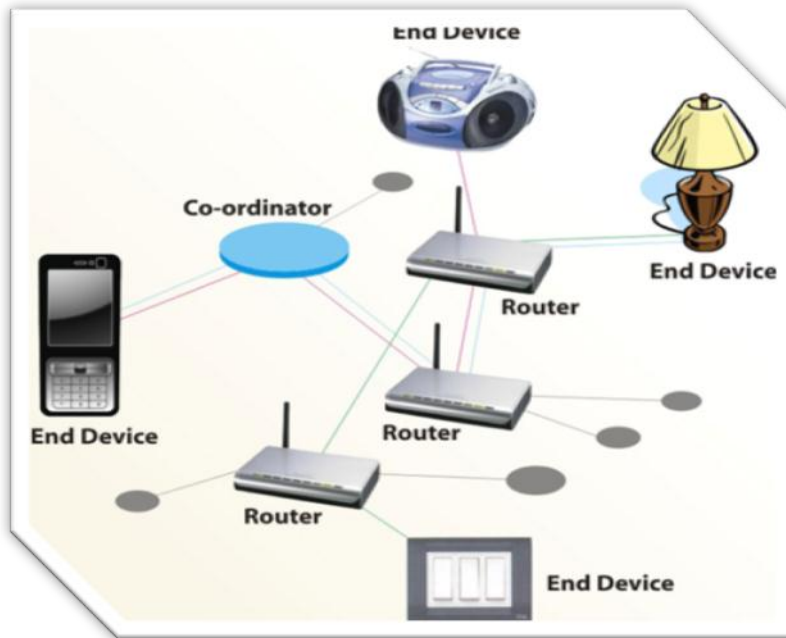
As well as running an application function, a router can act as an intermediate router, passing on data from other devices.

End Devices:

Contain just enough functionality to talk to the parent node (either the coordinator or a router) but they cannot relay data from other devices. This relationship allows the node to be asleep a significant amount of the time thereby giving long battery life.

4.3 Background Information

This Smart Meter Application design is based on a low-cost and low-data rate wireless technology that consists of three main devices to incorporate the ZigBee radio application.



In a ZigBee specification, three kinds of devices exist that incorporate ZigBee radios. They are:

- A) Coordinator: This organizes the network and maintains routing tables.
- B) Routers: These can talk to the coordinator, to other routers and to reduced-function end devices.
- C) Reduced-function End-devices: These can talk to routers and the coordinator, but not each other.

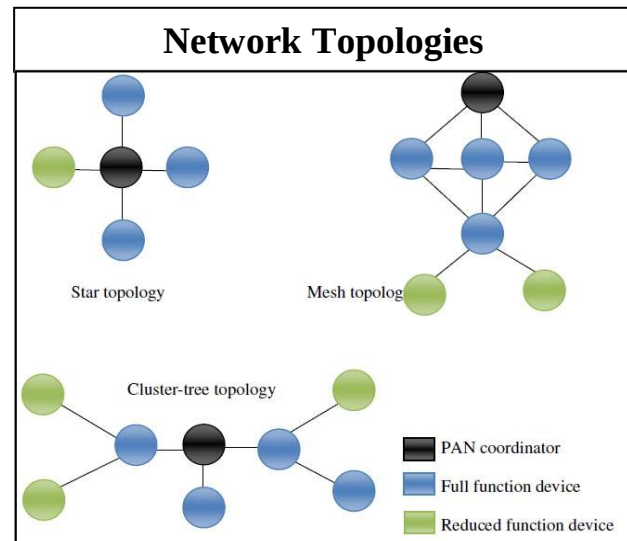
Routers and the coordinator remain powered and do not go to sleep, while end devices wake up only when they need to communicate and immediately go back to sleep to minimize power consumption and promote long battery life in battery-powered devices.

5.0 – System Summary

5.1 System Overview

ZigBee technology has three different network topology configurations: star, tree, and mesh topologies.

In a star topology, the network controlled by one single device is the ZigBee coordinator. All other devices, known as end devices, directly communicate with the coordinator.



In mesh and tree topologies, the network may be extended with ZigBee routers. Mesh networks allow full peer-to-peer communication. This specification describes only intra-PAN networks, that is, networks in which communications begin and terminate within the same network

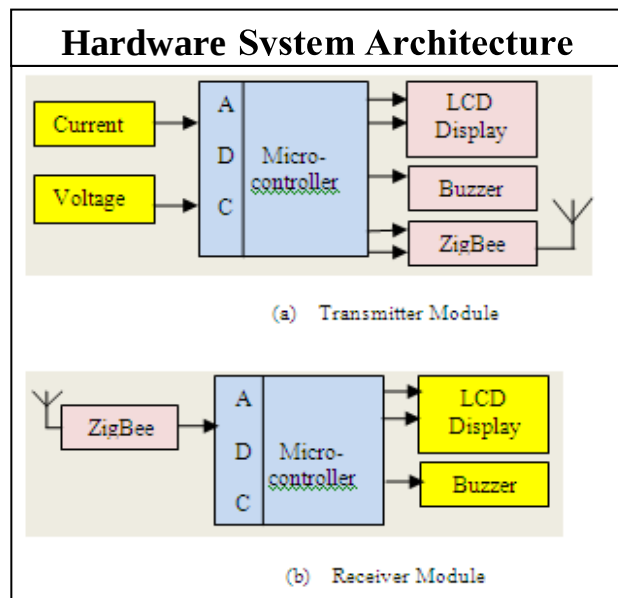
Looking to the need of trustworthy power monitoring system, a ZigBee modeled system has to be comprised of the following modules:

Transmitter module

Transceiver module

5.2 System Configuration

The setup consists of the following elements; end devices, microcontroller, LCD display, and ZigBee module. The Microcontroller is used for the function of data processing, data storage, human interfacing and interoperability with the external environment.



For this network setup, we chose the controllers and developments tools of Microchip Technology. Module of XB PRO Series2B of DIGI will be used here as a part of ZigBee stack which will then be addressed by mixed API mode with hardware controlled flow control for USART interfacing with the desired baud rate. ZigBee protocol is for wireless communication between transmitter and transceiver module. The LCD display displays the current and voltage parameters.

5.3 Bill of Materials:

No.	Product Code	Product Name	Qty	Internet Price (USD)	Retail Price (USD)	Amount (USD)
1	SK40C	Enhanced 40 pins PIC Start-Up Kit	1	12.87	14.52	12.87
2	DS-LCD-162A	LCD (16x2)	1	6.77	8.06	6.77
3	BD-BB-0508	Breadboard (Small)	1	2.74	3.00	2.74
4	UIC00B	USB ICSP PIC Programmer V2010	2	16.42	18.71	32.84
5	IC-PIC-18F4550	IC PIC18F4550	1	8.06	9.68	8.06
6	AD-TMC-500PM	Adapter Teletron TMC-500PM	1	5.81	6.45	5.81
7	WR-JW-MM65	Male to Male Jumper Wire	1	5.81	6.45	5.81
8	SKXBEE-PRO	XBEE PRO STARTER KIT	2	85.48	94.19	170.97

Total Amount : 245.87

You save this with internet payment : 28.10

Payable Amount : 245.87

- 0.00

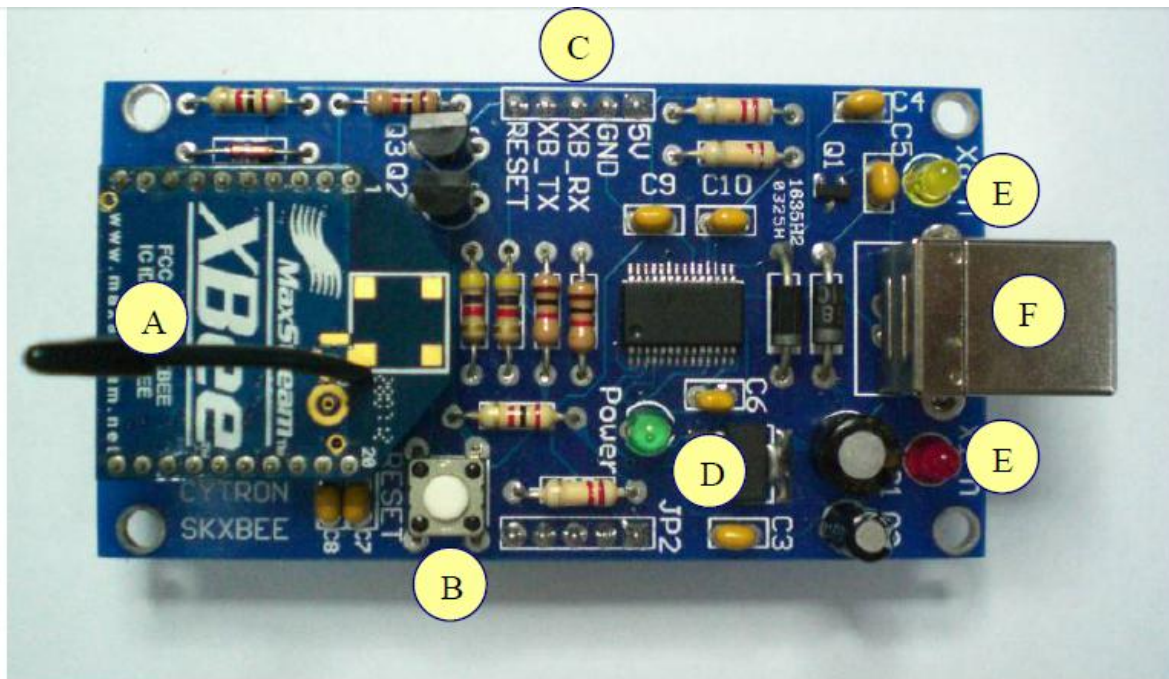
Shipping Fees : 30.16

Grand Total : 276.03

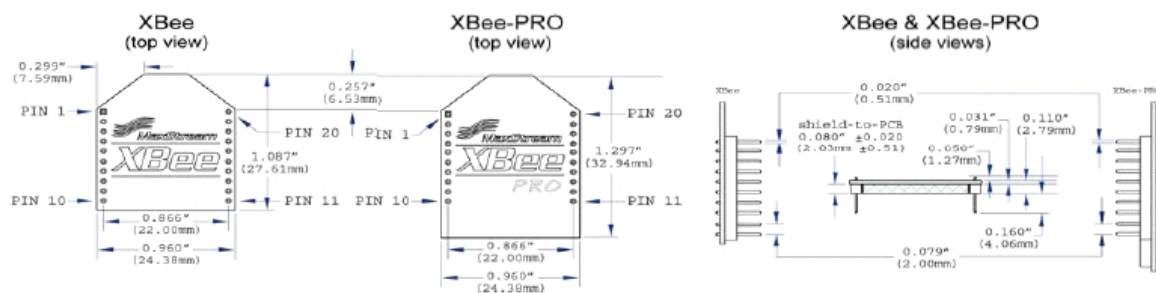
Item to Purchase	Qty.	Price after Options	Ext. Price
XBee Smart Plug Item#: XR-Z14-CW2P6	2	\$84.00	\$168.00
Subtotal:			\$444.03
Item to Purchase	Qty.	Price after Options	Ext. Price
Arduino UNO	1	\$25.95	\$168.00
Subtotal:			\$469.98

6.0 – Getting Started

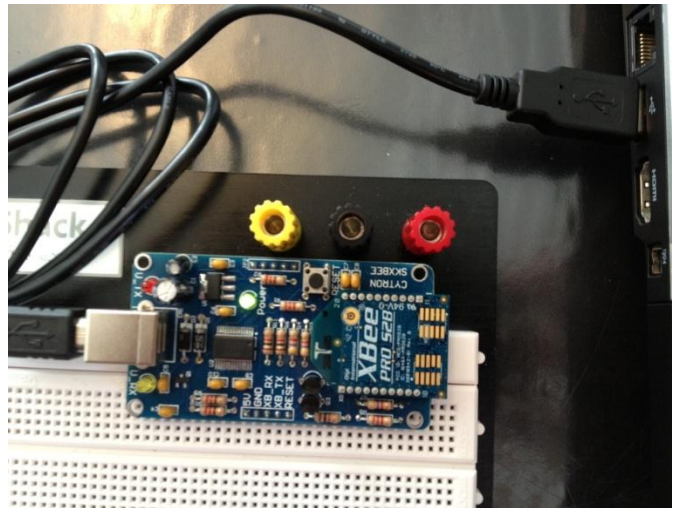
6.1 Enabling XBee



Label	Function
A	XBee module.
B	On board reset button for XBee module.
C	5 ways header pin for external power and interface to microcontroller.
D	On board 3.3V power indicator LED. It is green color.
E	Two LED indicators for USB's transmitter and receiver status.
F	USB B type socket.



a. Connect USB cable (B type) to the SKXBee USB socket, and the other end (A type) to a PC as shown in following figure.



b. Follow driver installation guide.

c. SKXBee is ready, once plugged into the PC.

d. Use X-CTU, software used to communicate with XBee, from Digi International.

First, setup X-CTU to change configuration or transmit data.

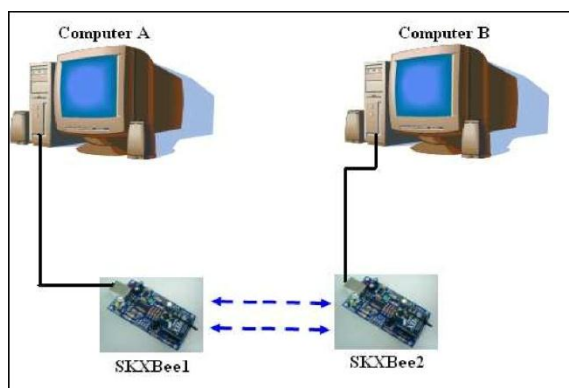
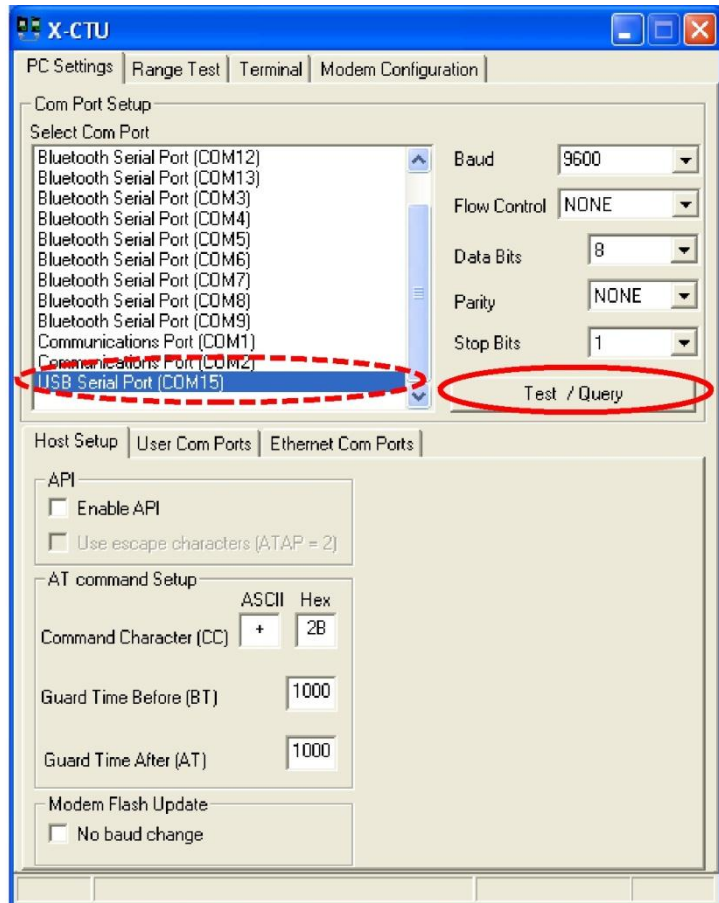


Install the X-CTU Software to the PC by double clicking on “setup_x-ctu.exe” file downloaded from the SKXBee website. X-CTU is

configured to enable the communication and display data from XBee module. Please follow the instruction as stated below to setup the computer.

e. In X-CTU Software click on “PC Settings” tab. Verify the settings to match those in XBee Module (as parameter in the left picture).

- f. Choose the COM Port and click ‘Test/Query’ tab. Then, click ‘OK’.
- g. To modify or read XBee Module parameters use AT command Mode.
- h. First, select the ‘Terminal’ tab on X-CTU Software. Enter AT Command Mode and send the 3-character command sequence ‘+++’ and XBee module will response ‘OK’ which means that it has already enter Command Mode.
- i. If a command is successful the module will return an “OK” message, otherwise the module will return an “ERROR” message.
- j. To exit AT Command Mode, send ATCN Command. CT (Command Mode Timeout) Command, the XBee Module automatically returns to Idle Mode. Therefore, if another command needs to be sent you must return to Command Mode.



- k. To communicate between two SKXBee, example TRANSMITTER and TRANSCIEVER, Source Address (MY) of both XBee1 and XBee2 must be match to Destination Address (DL) of both XBee1 and XBee2.

l. Set the address for each module as shown below.

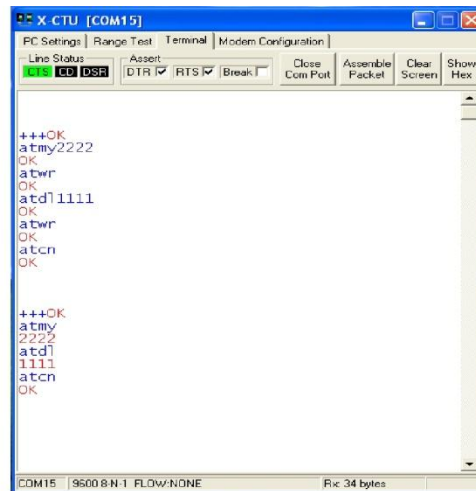
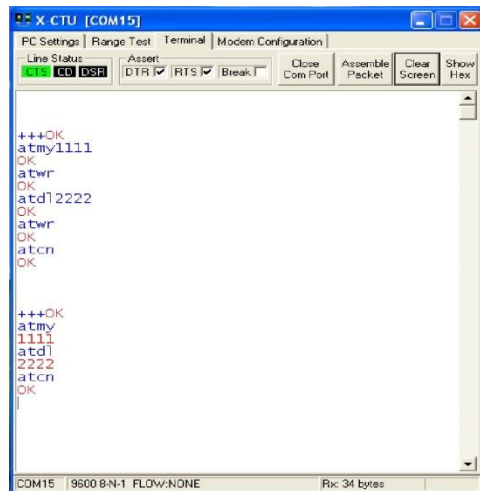
SKXBee1 : MY = 1111

SKXBee2

MY = 2222

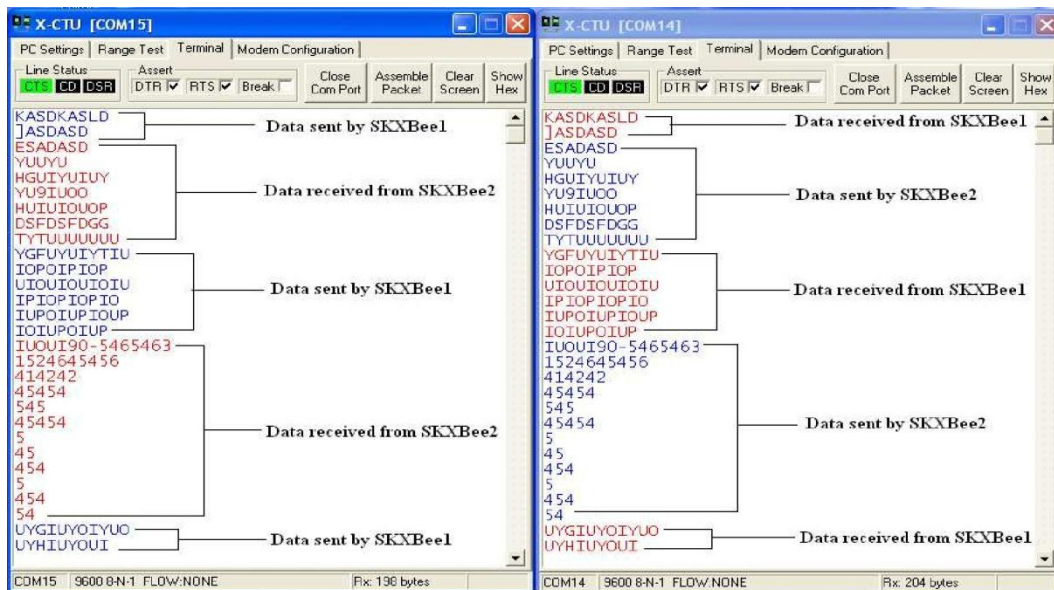
DL = 2222

DL = 1111



o. After building the connection between the two XBee they will communicate among each other.

p. Below is an example of data sent and received of XBee1 and XBee2 in X-CTU terminal.



q. To restore XBee Module defaults setting, click 'Restore' button.

r. To modify XBee parameters refer to Command Reference Tables in Data Sheet; you can modify them by using ATCT (Command Mode Timeout).

6.2 Setting Up a Network

Our network consists of two XBee Pro S2B modules and two Digi XBee Smart Plugs

When the XBee Pro S2B modules are connected with the USB, the PAN ID, Scan Channels, and Baud Rate must be set the same for each.

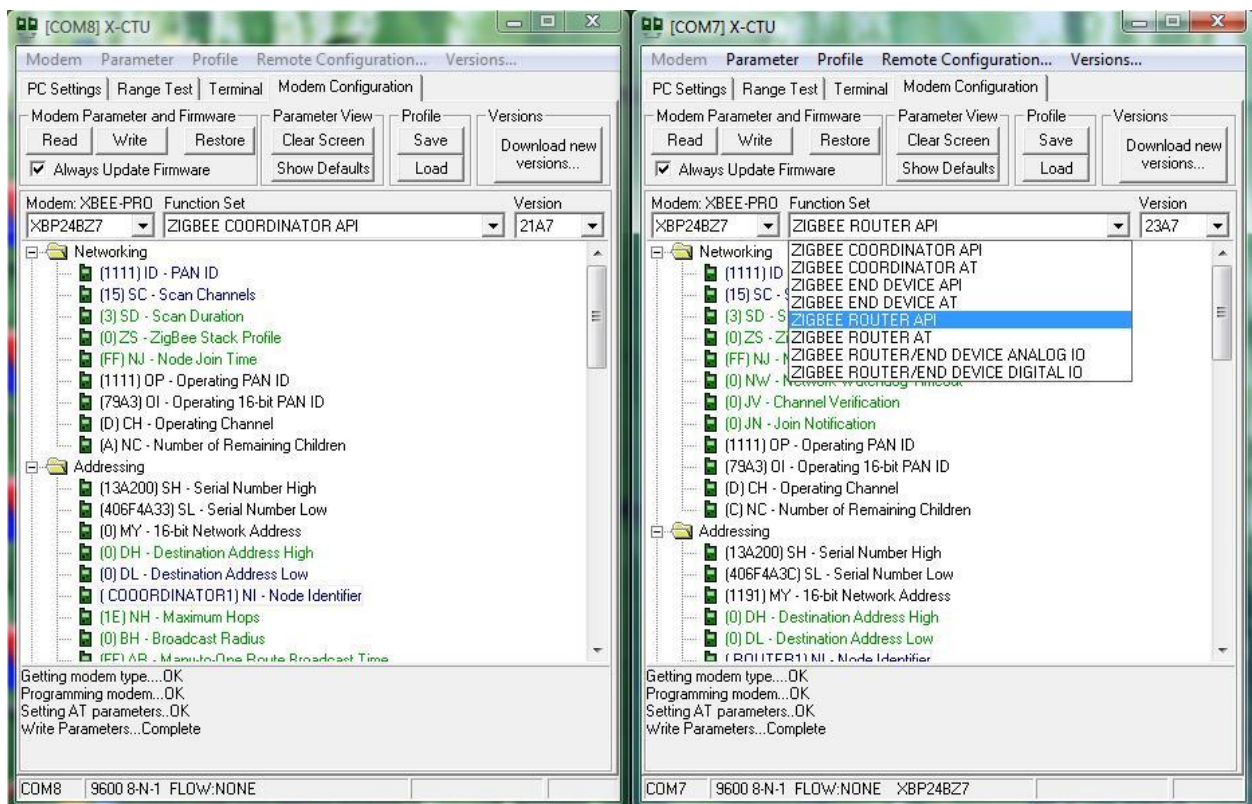
PAN ID = 1111

Scan Channels = 15

Baud Rate = 9600

The modules also must be set as coordinator, router, or end node.

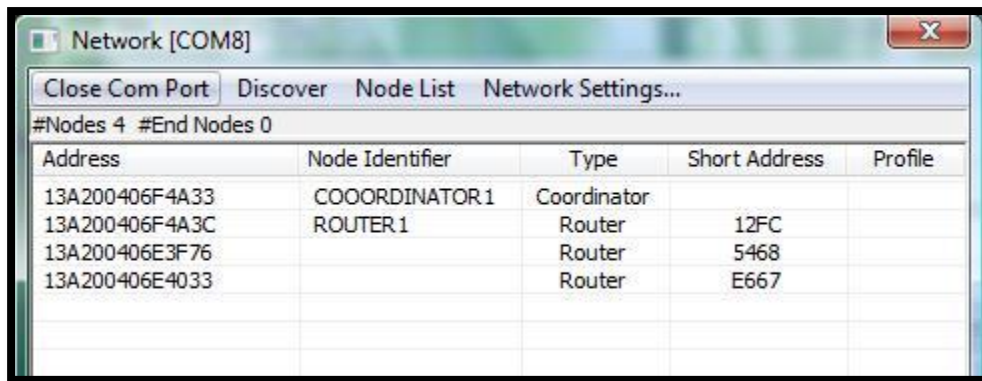
Under the Function Set drop-down menu, select ZIGBEE COORDINATOR API for one and ZIGBEE ROUTER API for the other.



To check the network for all its devices click Remote Configuration..., then Open Com Port, then Discover Devices.

All devices that are in the network will show up with their address, name, and type of node.

The address is needed to send commands in API mode



6.3 API Mode

Using API mode makes it possible to send commands from a transmitting radio to a receiving radio. For the Smart Meter application is to toggle an IO on a remote radio from a high to low state.

- One radio will operate as a Base and the other as a Remote.
- API must be enabled on the Base Radio using the check box in X-CTU.
- The 64-bit addresses obtained from the Device discovery are needed to create the API frame for the receiving radio (Base).
- The following command is to set DIO4 high (Turn Smart Plug ON)

7E 00 10 17 05 00 13 A2 00 40 6E 3F 76 FF FE 02 44 34 05 4F

7E	Start Delimiter
00 10	Number of bytes – 2 (Start and checksum)
17	API Identifier
05	Identifies the UART Date Frame
00 13 A2 00 40 6E 3F 76	64 Bit Address of Receiving Device
FF FE	16-bit address of the destination device
02	Remote Command Options
44 34	Name of Command
05	Requested parameter value to set the given register
4F	Checksum

The name of the command is 44 34 to turn the smart plug on.

44 is the hex equivalent of ASCII character D

34 is the hex equivalent of ASCII character 4

The setting for D4 is 05 (the hex equivalent of decimal 5)

D4 is set to 5 to turn on the smart plug and 4 to turn it off.

Calculating the CHECKSUM:

The checksum can be calculated by following these steps:

1. Add all the hex digits of the frame except for the first three bytes (Start Delimiter and Length Bytes)
2. Take the last two digits of the sum.
3. Subtract these two digits from the hex value FF.

Here are our commands before calculating the checksum. The checksum is blank.

- 1) 7E 00 10 17 05 00 13 A2 00 40 6E 40 33 FF FE 02 44 34 05 __
- 2) 7E 00 10 17 05 00 13 A2 00 40 6E 3F 76 FF FE 02 44 34 05 __
- 3) 7E 00 10 17 05 00 13 A2 00 40 6E 40 33 FF FE 02 44 34 04 __
- 4) 7E 00 10 17 05 00 13 A2 00 40 6E 3F 76 FF FE 02 44 34 04 __

Checksum Calculations:

- 1) $17+5+0+13+A2+0+40+6E+40+33+FF+FE+2+44+34+5 = 46E$

$$\text{Checksum 1} = FF - 6E = 91$$

- 2) $17+5+0+13+A2+0+40+6E+3F+76+FF+FE+2+44+34+5 = 4B0$

$$\text{Checksum2} = FF - B0 = 4F$$

- 3) $17+5+0+13+A2+0+40+6E+40+33+FF+FE+2+44+34+4 = 46D$

$$\text{Checksum3} = FF - 6D = 92$$

- 4) $17+5+0+13+A2+0+40+6E+3F+76+FF+FE+2+44+34+4 = 4AF$

$$\text{Checksum 4} = FF - AF = 50$$

These are the commands for powering the two smart plugs:

Each Smart Plug has its own API commands. The command must be edited with the correct address and the correct, calculated check sum

Command to set DIO4 high (Smart Plug ON):

Smart Plug 1: 7E 00 10 17 05 00 13 A2 00 40 6E 40 33 FF FE 02 44 34 05 91

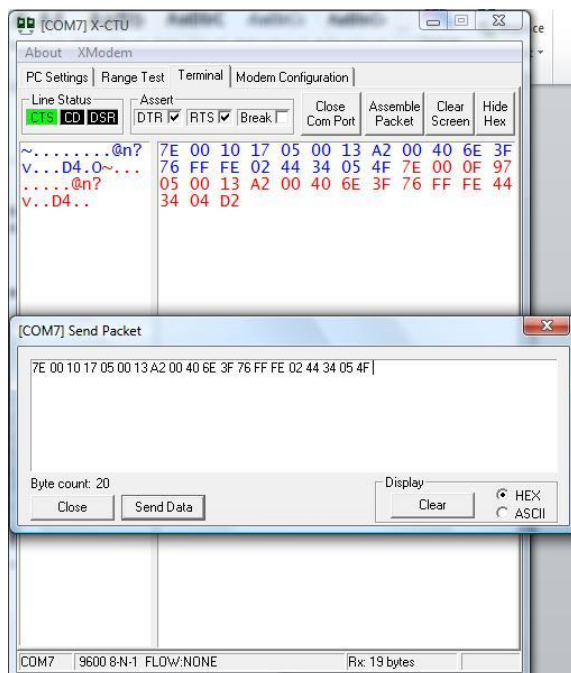
Smart Plug 2: 7E 00 10 17 05 00 13 A2 00 40 6E 3F 76 FF FE 02 44 34 05 4F

Command to set DIO4 Low (Smart Plug OFF):

Smart Plug 1: 7E 00 10 17 05 00 13 A2 00 40 6E 40 33 FF FE 02 44 34 04 92

Smart Plug 2: 7E 00 10 17 05 00 13 A2 00 40 6E 3F 76 FF FE 02 44 34 04 50

Sending the command in X-CTU:



Under the Terminal tab, click show hex and then Assemble Packet.

Check HEX in the bottom right of the window and type the API frame. After it is all typed, click Send Data.

The sent packet will show up in blue letters and a response will show up in red indicating that the API frame contains no errors and it was successfully sent.

The API frame we sent will turn on the smart plug.

6.4 Digi XBee Smart Plug

The smart plug acts as a router. An XBee device can join the network that we set up if it is in range of the smart plug.



The AT commands for the XBee Smart Plug's features are:

Light sensor

D1=2 (Analog input)

Current sensor

D3=2 (Analog input)

Power Relay

D4=4 (Turns outlet off)

D4=5 (Turns outlet on)

Associate LED (Green):

ON when plugged in

Blinks when the plug joined the network

Outlet LED (Yellow):

On when the User outlet is powered on

Reset Button:

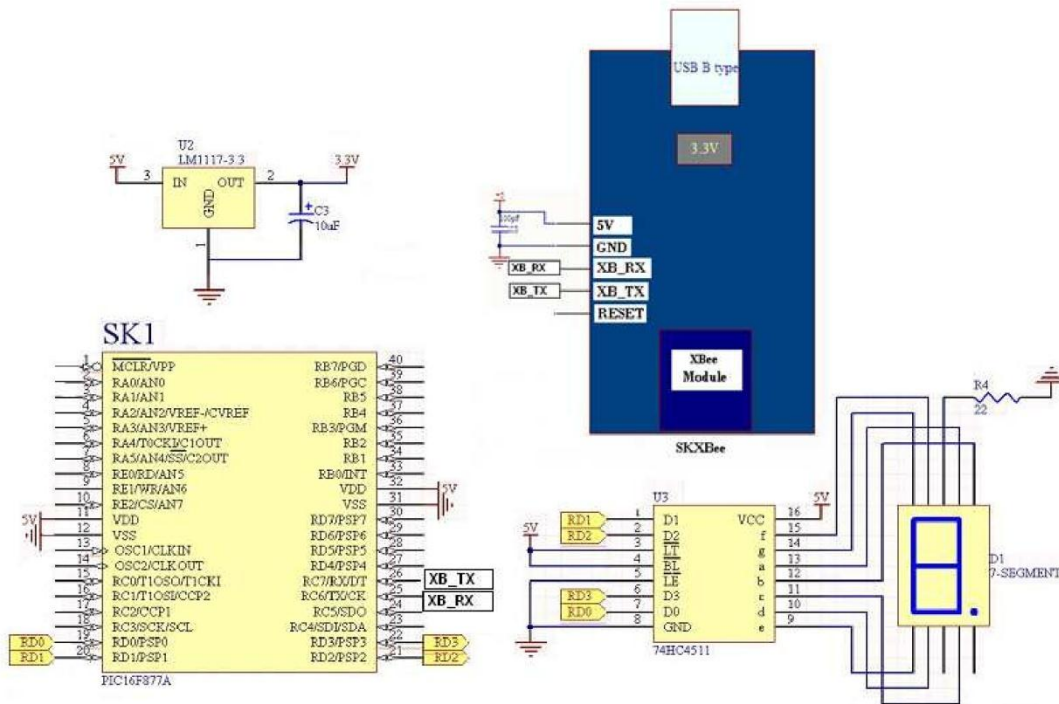
Press the reset button four times to reset the smart plug and have it join an available network

7.0 – Methods and Procedure

7.1 Hardware (Construction) Phase

How to set up the SK40C and PIC as a coordinator and router simulator

- a. Setup SK40C with PIC.
 - Prepare a PCB board and all the components required according to the schematic.



- b. Load programs to the microcontroller using a PCI interface.

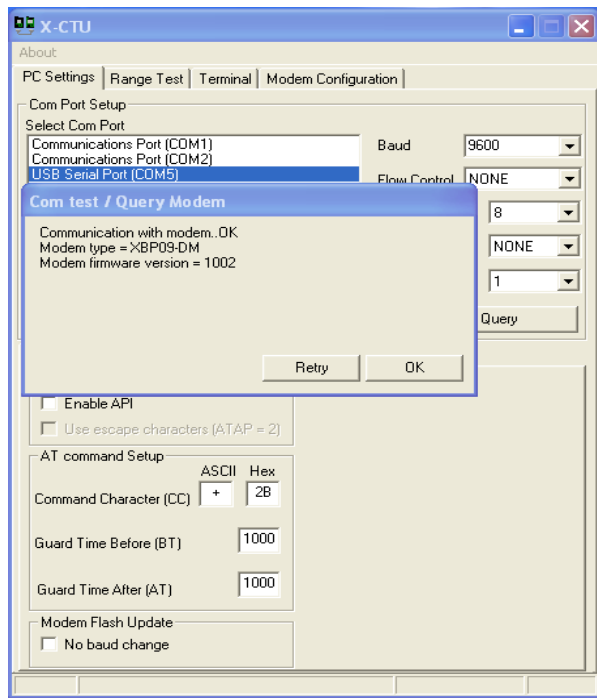
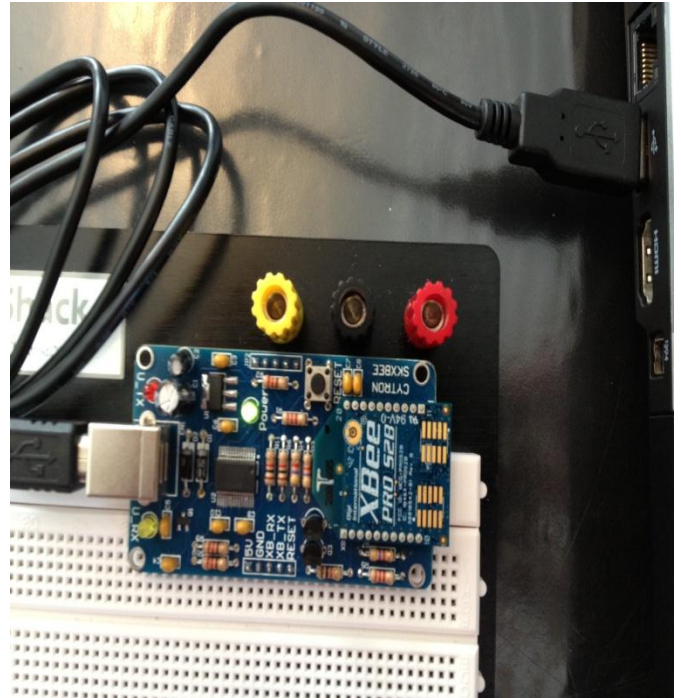
Sample source codes are downloaded from the web.

```
void init(void) // subroutine to initialize
{
    SPBRG=129; // set baud rate as 9600 baud
    BRGH=1;
    TXEN=1;
    CREN=1;
    SPEN=1;
    TRISD = 0b00000000;
    seg = 0b00000000;
}
```

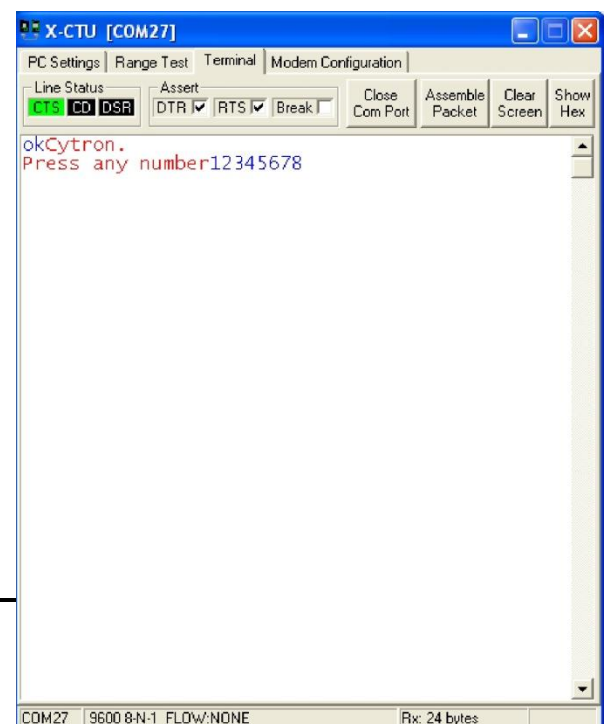
- c. Turn on, press and release reset button of SK40C.
- d. After setting everything up, the SK40C will be able to receive data through the network.

7.2 Using SKXBee as a Coordinator

- a. Connect SKXBee to PC using USB cable as the picture below shows. Set the address for each module, test the communication and go to 'Terminal' setting tab.

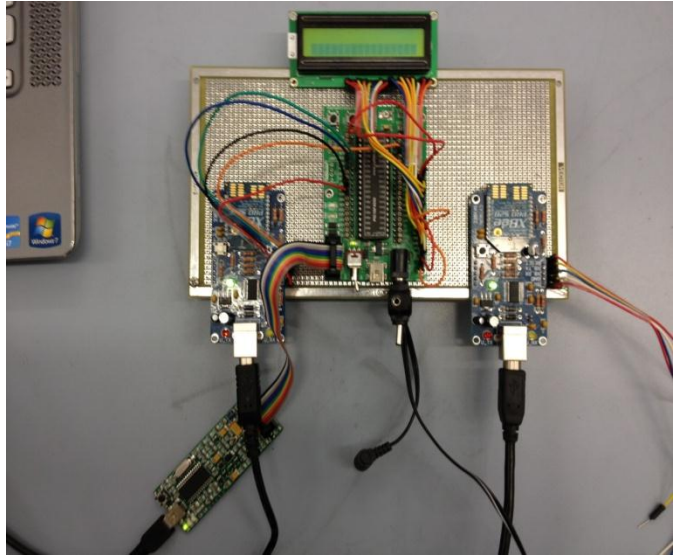


- b. Press reset button of SK40C, followed by pressing "ok"



and “Press any number” will appear as shown in picture to the right. Connect end devices. Red color is for data received while blue color is for data sent.

- c. Energy consumed will appear in the LCD screen which is shown below.



7.3 Modification

How to set up the ArduinoUNO as a coordinator and router simulator with LCD screen:

- a. Using a USB cable, connect the ArduinoUNO board with computer.
- b. Connect the LCD screen with the ArduinoUNO by following the pin-out diagram shown below

LCD PIN	CONNECT TO
1 (Vss)	GND Arduino Pin
2 (Vdd)	+5V Arduino Pin
3 (Contrast)	Resistor or Potentiometer to GND Arduino Pin
4 (Rs)	Arduino Pin 12
5 (R/W)	Arduino Pin 11
6 (Enable)	Arduino Pin 10
7 (No Connection)	
8 (No Connection)	
9 (No Connection)	
10 (No Connection)	
11 (Data 4)	Arduino Pin 5
12 (Data 5)	Arduino Pin 4
13 (Data 6)	Arduino Pin 3
14 (Data 7)	Arduino Pin 2
15 (Backlight)	Resistor to Arduino Pin 13
16 (Backlight GND)	GND Arduino Pin

- c. Connect the ArduinoUNO with the XBee using a breadboard and following the pin-out diagram shown below

XBee	Arduino
Vcc or 3.3V	3V3
TX or DOUT	RX or 0
RX or DIN	TX or 1
GND	GND

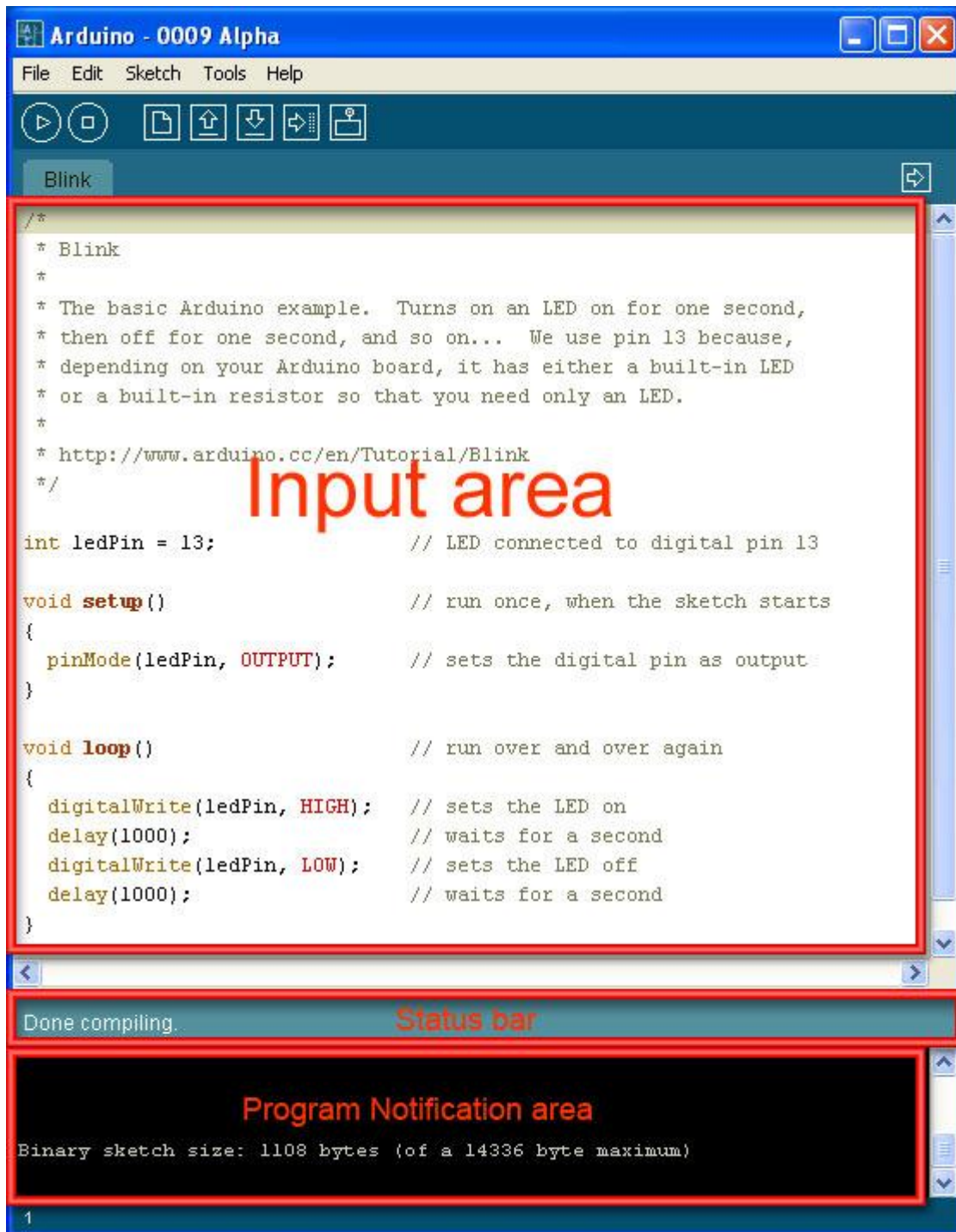
7.4 Input Procedure and Expected Output

- a. Open Arduino workspace



- b. Select the correct microcontroller and COM port and then begin writing program. After select verify/complete

- c. After compiling the screen breaks down as shown below



The Arduino Uno uses code written in C language to communicate with the XBee module via UART, and commands such as `Serial.print`, `Serial.write`, and `Serial.begin` simplify serial communication software. The Arduino Uno module always assumes 8-bit transmission with no parity and one stop-bit, abbreviated as 8N1.

The following C language program will monitor the Arduino Uno UART input for a new byte received from the XMTR XBee module. When the XBee receives a byte, it will save it in array that can hold five values. After the SKBee has received five bytes, it will send them back to the XMTR module that will transmit wirelessly to the RCVR module. The RCVR module will send the bytes to the PC, which displays them in the terminal emulator X-CTU.

```
/* Final Project Program
```

```
Piter Garcia, 05/02/2012
```

```
*/
```

```
#include <SoftwareSerial.h>
```

```
//#include <XBee.h>
```

```
char charrcvd[5];           // Set up array for rcvd bytes
```

```
int counter;                // Set counter variable
```

```
void setup()                // Set Serial port for 9600
```

```
{                            // bits/sec
```

```
    Serial.begin(9600);
```

```
}
```

```
void loop()                 // main loop starts here
```

```
{
```

```
    counter = 0;            // set counter to zero to start
```

```
    while(counter < 5)      // while count is less than 5
```

```
    {                      // go through this loop
```

```
        if (Serial.available() > 0)    // check UART for any new data
```

```
        {                      // detected new UART
```

```
            charrcvd[counter] = Serial.read();    // detected new UART
```

```
            counter = counter + 1;                // data, so get
```

```
                                                    // the UART BYTE
```

```
        }                                          //increment counter by one
```

```
}
```

```

// received five characters,
// so exit while loop
counter = 0; // set counter to zero to start
while(counter < 5) // while counter is less than 5
{ // go through this loop
  Serial.write(charrcvd[counter]); // print character
                                     // as a byte code
  delay(100); // wait 100msec between
               // characters
  counter = counter * 1; // increment counter by one

  Serial.print("\n\r"); // "print" new line
} // printed five characters
  // so exit while loop
}

//end of code, but execution returns to loop()
```

The Serial.begin command sets the Arduino Uno UART data rate at 9600 bits/second. The first while (counter < 5) loop will save five bytes received from the UART and this loop include the statement:

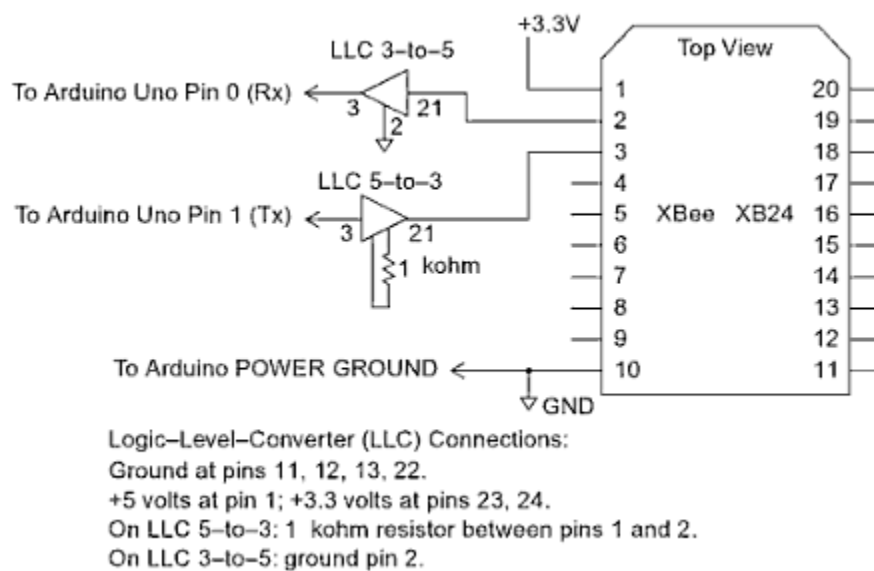
If (Serial.available () > 0)

The program needs this statement to determine whether the UART has a new byte or not. The Serial.available() function returns the value of 0 if UART receiver is empty and it returns a non-zero value when the UART has a new byte available. Without this statement, the software would

continuously read the UART receiver, regardless of whether or not it contained new data, and cause the program to display whatever byte the UART receiver holds. An Arduino program must check the UART for available data before it tries to read from it. Other MCUS behave in a similar fashion; that is, they provide a “flag” that software can test to determine if a UART or other device is ready or not ready, full or empty, on or off, and so on.

This program will save bytes in the `charrcvd` array and then transmit them back to the XMTR module so they appear via wireless link in the terminal-emulator X-CTU. The `delay` command provides a 100-millisecond (0.1 second) delay so you can see the five characters arrive on the screen one by one.

The schematic below shows the connections from the Arduino Uno module to logic-level-converter circuits, and to XBee module. Note the use of separate logic-conversion circuit for 5-V to 3-V logic and for 3-V to 5-V logic.



```
/*
  Smart Meter Home Application
  API frame software development
*/

#include <XBee.h>
#include <SoftwareSerial.h>
#include <LiquidCrystal.h>

LiquidCrystal lcd(12, 11, 10, 5, 4, 3, 2);
int backLight = 13;  // pin 13 will control the backlight

/*
Series 2 XBee
Sends a ZB TX request with the value of analogRead (pin5) and checks the status response for success
*/

// creating the XBee object
XBee xbee = XBee();
//Command to set DIO4 Hi (Smart Plug ON):

//Smart Plug 1: 7E 00 10 17 05 00 13 A2 00 40 6E 40 33 FF FE 02 44 34 05 91
//Smart Plug 2: 7E 00 10 17 05 00 13 A2 00 40 6E 3F 76 FF FE 02 44 34 05 4F

//Command to set DIO4 Low (Smart Plug OFF):

//Smart Plug 1: 7E 00 10 17 05 00 13 A2 00 40 6E 40 33 FF FE 02 44 34 04 92
//Smart Plug 2: 7E 00 10 17 05 00 13 A2 00 40 6E 3F 76 FF FE 02 44 34 04 50

uint8_t payload[] = {'7E','00','10','17','05','00','13','A2','00','40','6E','40','33','FF','FE','02','44','34',
'05','91'};

// SH + SL Address of receiving XBee
XBeeAddress64 addr64 = XBeeAddress64(0x0013A200, 0x406F4A33);
ZBTxRequest zbTx = ZBTxRequest(addr64, payload, sizeof(payload));
ZBTxStatusResponse txStatus = ZBTxStatusResponse();


int pin5 = 0;
int statusLed = 13;
int errorLed = 13;
```



```
void flashLed(int pin, int times, int wait)
{
  pinMode(backLight, OUTPUT);
  digitalWrite(backLight, HIGH); // turn backlight on. Replace 'HIGH' with 'LOW' to turn it off.
  lcd.begin(16,2);           // columns, rows. use 16,2 for a 16x2 LCD, etc.
  lcd.clear();               // start with a blank screen
  lcd.setCursor(0,0);        // set cursor to column 0, row 0 (the first row)
  lcd.print("Home Meter App."); // change this text to whatever you like. keep it clean.
  delay(1000);

  for (int i = 0; i < 2; i++)
  {
    digitalWrite(10, HIGH);
    delay(1000);

    digitalWrite(10, LOW);
    if (i + 1 < 2)
    {
      delay (1000);
    }
  }
}
```

```
void setup()
{
  flashLed(10, 2, 1000);
  digitalWrite(backLight, HIGH); // turn backlight on. Replace 'HIGH' with 'LOW' to turn it off.
  lcd.begin(16,2);           // columns, rows. Use 16,2 for a 16x2 LCD, etc.
  lcd.clear();               // start with a blank screen
  lcd.setCursor(0,0);        // set cursor to column 0, row 0 (the first row)
  lcd.print("Dig Pins Written"); // change this text to whatever you like. keep it clean.
  delay(1000);

  pinMode(statusLed, OUTPUT);
  pinMode(errorLed, OUTPUT);

  xbee.begin(9600);

  digitalWrite(backLight, HIGH); // turn backlight on. Replace 'HIGH' with 'LOW' to turn it off.
```

```
lcd.begin(16,2);      // columns, rows. use 16,2 for a 16x2 LCD, etc.
lcd.clear();          // start with a blank screen
lcd.setCursor(0,0);   // set cursor to column 0, row 0 (the first row)
lcd.print("Setting Up XBee"); // change this text to whatever you like. keep it clean.
delay(1000);

loop();
}

void loop()
{
  // break down 10-bit reading into two bytes and place in payload
  pin5 = analogRead(5);
  payload[0] = pin5 >> 8 & 0xff;
  payload[1] = pin5 & 0xff;

  xbee.send(zbTx);

  digitalWrite(backLight, HIGH); // turn backlight on. Replace 'HIGH' with 'LOW' to turn it off.
  lcd.begin(16,2);      // columns, rows. use 16,2 for a 16x2 LCD, etc.
  lcd.clear();          // start with a blank screen
  lcd.setCursor(0,0);   // set cursor to column 0, row 0 (the first row)
  lcd.print("Frame Sent"); // change this text to whatever you like. keep it clean.
  delay(1000);

  // flash TX indicator
  flashLed(statusLed, 1, 100);

  // after sending a tx request, we expect a status response
  // wait up to half second for the status response
  if (xbee.readPacket(500))
  {
    // got a response
    // should be a znet tx status
  }
}
```

```
if (xbee.getResponse().getApiId() == ZB_TX_STATUS_RESPONSE)
{
    xbee.getResponse().getZBTxStatusResponse(txStatus);

    // get the delivery status, the fifth byte
    if (txStatus.getDeliveryStatus() == SUCCESS)
    {
        // success. time to celebrate
        flashLed(statusLed, 5, 50);
    }
    else
    {
        // the remote XBee did not receive our packet. Make sure power is on.
        flashLed(errorLed, 3, 500);
    }
}
else if (xbee.getResponse().isError())
{
    // nss.print("Error reading packet. Error code: ");
    //nss.println(xbee.getResponse().getErrorCode());
}
else
{
    // local XBee did not provide a timely TX Status Response -- should not happen
    flashLed(errorLed, 2, 50);
}

delay(1000);
}
```

In the above program we use the AT command set and API packets to communicate with remote XBee and Smart plug modules knowing their addresses. The XBee module have an AT command mode time out parameter preset for 100 milliseconds. That setting means after an XBee module receives an AT command, it waits for 100 milliseconds before it exits the AT command mode. During the 100 milliseconds AT command Mode time out period, the state of the D4 will not change. The change occurs only after the time out period ends.

For a better understanding of the code, I send a signal to the LCD for every process done by the program to send the packet to the smart plug given address to turn on or off the end device. The tree main functions of these programs are the following: flashLed, setup, and loop. The first function is to communicate with the Arduino Uno with the LCD display in order to know that is running correctly and that the frame has been sent successfully. The second functions are the main function of the program. This method call the others method after initializing the XBee and sending the frame to a given address. The last function mentioned is to check that the Arduino Uno has received an answer from the XBee module. In this way, it knows whether it received the frame, and if it did not it will go back into a loop until it gets an answer from the XBee module.

8.0 – Schedule of Progress

8.1 First Quarter

In the beginning of the project, we were researching RF transceivers that support ZigBee to use for the Smart Meter Application. We had found transceivers from Atmel, Freescale, Texas Instruments and Microchip. These transceivers were sufficient for the project but they were not Dual In-line Packages (DIP). They would need soldering but we wanted the chip to be mountable for convenience and ease. In the end, we chose Digi's XBee-Pro RF Module. This module comes with a wire antenna that is necessary from transmitting and receiving data in the Smart Meter Application. It is also interoperable with other ZigBee devices.

With the XBee-Pro module, we chose the Cytron Technologies XBee Starter Kit. This kit is used to program the RF module with a microcontroller and power the RF module. The XBee-Pro is 3.3V power so the Starter Kit enables it to operate under 5V. When the module is connected to the starter kit, it is ready to connect to the microcontroller and PC.

The microcontroller we chose was PIC18F4550. This is a Serial Port Interface (SPI) microcontroller, which is ideal for monitoring applications that require periodical communications.

A startup kit and programmer are needed to program and use the microcontroller. We chose the PIC Start-Up Kit and PIC Programmer from Cytron Technologies. The PIC programmer and PIC Start-Up Kit are used to load and modify the program from a PC using a USB cable. When using the Start-Up Kit it is easy to plug the input and output components to the microcontroller. All of these parts for the project were ordered.

8.2 Second Quarter

Since the last progress report, we received all the parts we need for the Smart Meter Application. We have two XBee Pro modules with SK XBee boards. The SK XBee boards are used to power the XBee RF Module and program it with the microcontroller. The microcontroller we have is PIC18F4550. We purchased a SK40 board and UIC00B board to program the microcontroller through the boards USB.

Our first step was to setup our XBee RF transceivers. We used X-CTU to set the XBee Module Source address and Destination address. First, we connected the SKXBee with the XBee module to the computers USB port without the microcontroller connected. Using the X-CTU program, we tested the RF Module. After the test was confirmed, we set the baud rate to 9600bps and went into command mode.

In the command mode, we checked the previous Destination ID address of the module and changed it to a new Destination ID address using “WRITE” command. The “atwr” command is used to store a value in long-term memory. When a command is successfully executed, an “OK” message will come up and if it fails, “ERROR” will come up. In order for the two XBee Modules to communicate to each other, we must set the destination and source addresses for each. When everything was set up, we were able to make them communicate with each other. We could send messages from one to the other instantly.

8.3 Third Quarter

As of my last progress report, we encountered a problem with our XBee RF modules. We were unable to get the Digi XBee Smart Plug to join the network that we created with our XBee RF modules. After calling Digi customer support, they told us that our RF modules are not compatible with the Digi XBee Smart Plug. The XBee RF modules that we had were XBee compatible for other devices but they were not compatible with XBee smart plugs. We had to choose new modules that were compatible with the smart plugs. We chose the XBee-Pro ZB S2B modules. These are similar to the module we had but the original we had were S1 (Series 1). Before we ordered, we confirmed with Digi customer support that they were compatible.

When we received the new XBee RF Modules, we had to put them on the Starter Kit boards. We had to unsolder the old XBee RF modules and solder the new ones on. After they were soldered on, we tested them with the program X-CTU and it was able to communicate with each of them.

We were able to make a network using one XBee RF module and setting it as the coordinator. We set the other as a router and plugged the smart plug in. In X-CTU, we had to set the same PAN ID, Baud Rate, and Scan Channels for each module. The “ASSC” light on the smart plug is supposed to flash when it joins a network. After writing all the settings to the XBee RF modules, the smart plug did not join the network. We pressed the button on the smart plug four times to reset it and after a few seconds, the light started to flash.

Now that the smart plug joined the network, we wanted to obtain its address using X-CTU. When we first tried to discover devices, nothing came up on the list. After research, we found that I had to set the XBee RF modules as a “COORDINATOR API” instead of “COORDINATOR AT.” Once we changed the coordinator and router to API and reset the smart plug, we attempted to discover the devices. The coordinator, router and smart plug showed up on the list with their addresses and type of device they are. We need the addresses to write our program to communicate to each device.

We are currently writing the program for our microcontroller to send commands to our XBee RF modules. In our program, we will be able to turn electrical devices on and off through the smart plugs. Our program will tell the XBee RF modules to turn off all of the devices connected to the network. We are writing our program in MPLab. The program will be loaded onto our PIC18F4550 microcontroller using our programming board. We have to connect our microcontroller to one of our XBee RF modules, which will be able to use the program to wirelessly communicate to other XBee RF modules and smart plugs.

10.0 – Results

We were able to set up a network with our XBee modules by setting one as a coordinator and the other as a router. Our Digi Smart Plugs joined the network when we reset them to search for networks. Using X-CTU we discovered all the devices in our network and acquired the addresses to the smart plugs. Using these addresses, we built API frames that are used to turn each smart plug ON and OFF. These commands toggle the D4 pin, which is necessary to give power to the smart plug.

On the other hand, we built software to receive signal from the XBee modules and the smart plugs as well. Furthermore, we were able to develop software that sends frames to the XBee modules by initiating the XBee and Arduino Uno board, and establishing a wireless communication through the UART pins on the boards. To make sure our programs work correctly, we established a connection between the Arduino Uno and the LCD to monitor every step of the program during the API sending frame process. In fact, we were able to prove that the program works and the Arduino Uno works through the LCD commands.

11.0 – Ethical Concerns

We consider some of the ethical issues that arise in the practice of software/electrical engineering. Our comments are guided by the cases described within the project. As a group our preliminary responses include comments on: the relation between software safety and ethics, informed consent and ethical decisions, the customer's responsibility for end product quality, internal performance standards and customer consent to inferior work, and techniques for addressing ethical issues.

The practicing software/electrical engineer engages in a social process during software/electrical production and thereby acquires obligations to: users, clients, customers, colleagues, supervisors, the organization for which we work, and the discipline of software/electrical engineering. The ethical problems faced by the software engineer involve: the end product, the process of developing that product, and the human interactions in the development of the product.

Our practicing software and electrical engineering has a great potential to affect the lives of others. The ethical concerns of the practicing software/electrical engineer include the types of issues that mainly affect our society, energy.

The User Interface

Our project is to write the first stage of a more efficient electrical system which will be used by homes and will save a considerable amount of money. As software/electrical engineer we are to develop a friendlier user interface which will display needed information. This information can be used to determine what electronic appliance needs to be shut off, and to control all electronic devices. The system and the interface are to contain all of the functionality described in the requirements.

System Modifications

The system manages all electronic appliances, saving energy and money. The system becomes aware of some things it can do that will better protect every electronic appliance from the threat of a lower or higher current. The system will request the network to make immediate modifications to this system through a coordinator to maintain a safe environment.

Professional Judgment

Our system is working on an integrated control system for homes. The system will gather end devices information daily. This information will be used by the coordinator; ordering their status to control all of their functions in a single corporation.

12.0 – Future Considerations

In future projects, it is recommended that extensive research be done when choosing the parts for a similar project. Every manual should be read to make sure all parts are compatible. We ran into the problem of our original XBee modules not being compatible with our Digi XBee Smart Plugs. We needed to purchase XBee Series 2 modules. This put a delay on our progress. When you are not very familiar with a project, a lot of research should be done ahead of time to make sure the deadline is met.

CONCLUSION

Conclusion

Wireless communication is a cheap and easy way to provide network communication at places where there is no wired infrastructure. ZigBee provides all suitable data rates as compared to other wireless technology and withstands suitably for all applications. ZigBee technology provides reliable protection for the operation of electric power systems and benefits for home and even industrial automation applications because of low cost deployment and redeployment. The Smart Meter ZigBee network application can be implemented into our daily used devices such as lamp, radiator, ventilation, and others. This Application can monitor every device by judging from the read out from the sensor. Devices like those mentioned could easily be combined together to form a large and complex network within a home and even an industry.

The Smart Meter Home Application does not only make life more comfortable but also save energy. The amount of energy used by specific appliances is monitored, and when necessary the appliance is shut off. This application is also able to automatically, shut off an appliance when too much energy is in use. In brief, the ZigBee power monitoring system application makes it possible for users to conserve electricity and save money.

REFERENCE

Project References

Pheysia. (2011, July 30). Cytron Technologies Official Website. Retrieved March 03, 2012, from XBee/XBee Pro Wireless Communicate with Microcontroller: <http://tutorial.cytron.com.my/2011/07/30/xbeexbee-pro-wireless-communicate-with-microcontroller/>

Raut, Archana R., and L. G. Malik. (2011). ZigBee Based Industrial Automation Profile for Power Monitoring System. *International Journal On Computer Science & Engineering*, 3(5), 2028-2033.

@NewTech. (2012, April 12). @NewTech. Retrieved April 12, 2012, from XBee-PRO 868: <http://www.anewtech.net/embedded-computer-ethernet-switch-features.php?parentProduct=2705>

Solution.com, M. E. (2012, April 12). *My Energy Solution.com*. Retrieved April 12, 2012, from Home Energy 101: Where Energy is Used...and Often Wasted!: <http://www.myenergysolution.com/home-energy-basics/home-energy-use.html>

Technology, S. (2012, April 12). *WIRELESS PAN*. Retrieved April 12, 2012, from EMBEDDED360:http://www.embedded360.com/wireless_pan/zigbee.htm

Raut, Archana R., and L. G. Malik. (2011). ZigBee Based Industrial Automation Profile for Power Monitoring System. *International Journal On Computer Science & Engineering*, 3(5), 2028-2033. Retrieved February 7, 2012